

Laboratory Work 1

ER-Diagram of the International Airport Database

Prepared by: Zangar Nurakhaev

0. System Description (Summary)

The Airport Database manages the core operations of an international airport: airports, airlines, scheduled and actual flights, passenger bookings and their change history, boarding passes, checked baggage and baggage inspection, passenger profiles, and passenger security checks. The design below follows the entities, attributes, and relationships described in the task's system description.

1. Identified Entities

#	Entity	Description
1	Airport	An airport location used for departures and arrivals.
2	Airline	A company operating flights.
3	Flight	A scheduled/actual flight operated by an airline between two airports.
4	Passenger	A person who books flights.
5	Booking	A reservation made by a passenger for a specific flight.
6	Booking_Change_History	A log of changes made to an existing booking.
7	Boarding_Pass	A boarding document issued for a booking.
8	Baggage	A baggage item registered for a booking.
9	Baggage_Check	The inspection result of baggage belonging to a booking.
10	Security_Check	The airport security screening result of a passenger.

2. Attributes per Entity

PK = Primary Key, **FK** = Foreign Key. Data types use a generic SQL notation (INT, VARCHAR(n), DATE, TIMESTAMP, DECIMAL(p,s)).

AIRPORT

Attribute	Data type	Key	Notes
airport_id	INT	PK	Uniquely identifies an airport
airport_name	VARCHAR(100)		Required
country	VARCHAR(50)		Required
state	VARCHAR(50)		Optional
city	VARCHAR(50)		Required
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

AIRLINE

Attribute	Data type	Key	Notes
airline_id	INT	PK	Uniquely identifies an airline
airline_code	VARCHAR(10)		Unique, required
name	VARCHAR(100)		Required
country	VARCHAR(50)		Required
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

FLIGHT

Attribute	Data type	Key	Notes
flight_id	INT	PK	Uniquely identifies a flight
airline_id	INT	<i>FK -> Airline.airline_id</i>	Required
departure_airport_id	INT	<i>FK -> Airport.airport_id</i>	Required
arrival_airport_id	INT	<i>FK -> Airport.airport_id</i>	Required
departing_gate	VARCHAR(10)		Optional
arriving_gate	VARCHAR(10)		Optional
scheduled_departure_time	TIMESTAMP		Required
scheduled_arrival_time	TIMESTAMP		Required
actual_departure_time	TIMESTAMP		Optional (set after departure)
actual_arrival_time	TIMESTAMP		Optional (set after arrival)
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

PASSENGER

Attribute	Data type	Key	Notes
passenger_id	INT	PK	Uniquely identifies a passenger
first_name	VARCHAR(50)		Required
last_name	VARCHAR(50)		Required
gender	VARCHAR(10)		Optional
date_of_birth	DATE		Required
country_of_citizenship	VARCHAR(50)		Required

country_of_residence	VARCHAR(50)		Optional
passport_number	VARCHAR(20)		Unique, required
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

BOOKING

Attribute	Data type	Key	Notes
booking_id	INT	PK	Uniquely identifies a booking
flight_id	INT	<i>FK -> Flight.flight_id</i>	Required
passenger_id	INT	<i>FK -> Passenger.passenger_id</i>	Required
status	VARCHAR(20)		Required (e.g. confirmed, cancelled)
booking_platform	VARCHAR(30)		Optional (e.g. web, mobile, agent)
ticket_price	DECIMAL(10,2)		Required
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

BOOKING_CHANGE_HISTORY

Attribute	Data type	Key	Notes
change_id	INT	PK	Uniquely identifies a change record
booking_id	INT	<i>FK -> Booking.booking_id</i>	Required
field_changed	VARCHAR(50)		Required (name of changed attribute)
old_value	VARCHAR(100)		Optional
new_value	VARCHAR(100)		Optional
changed_at	TIMESTAMP		Required, auto-set

BOARDING_PASS

Attribute	Data type	Key	Notes
boarding_pass_id	INT	PK	Uniquely identifies a boarding pass
booking_id	INT	<i>FK -> Booking.booking_id</i>	Unique, required (1 pass per booking)
seat	VARCHAR(5)		Required

boarding_time	TIMESTAMP		Required
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

BAGGAGE

Attribute	Data type	Key	Notes
baggage_id	INT	PK	Uniquely identifies a baggage item
booking_id	INT	<i>FK -> Booking.booking_id</i>	Required
weight_kg	DECIMAL(5,2)		Required
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

BAGGAGE_CHECK

Attribute	Data type	Key	Notes
baggage_checking_id	INT	PK	Uniquely identifies a baggage check
booking_id	INT	<i>FK -> Booking.booking_id</i>	Required
passenger_id	INT	<i>FK -> Passenger.passenger_id</i>	Required
check_result	VARCHAR(20)		Required (e.g. passed, flagged)
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

SECURITY_CHECK

Attribute	Data type	Key	Notes
security_check_id	INT	PK	Uniquely identifies a security check
passenger_id	INT	<i>FK -> Passenger.passenger_id</i>	Required
check_result	VARCHAR(20)		Required (e.g. passed, flagged)
created_at	TIMESTAMP		Required, auto-set
updated_at	TIMESTAMP		Auto-updated

3. Relationships and Cardinality

Cardinality is expressed in (min, max) notation at each end of the relationship: (0,1) = optional, at most one; (1,1) = mandatory, exactly one; (0,N) = optional, many; (1,N) = mandatory, many.

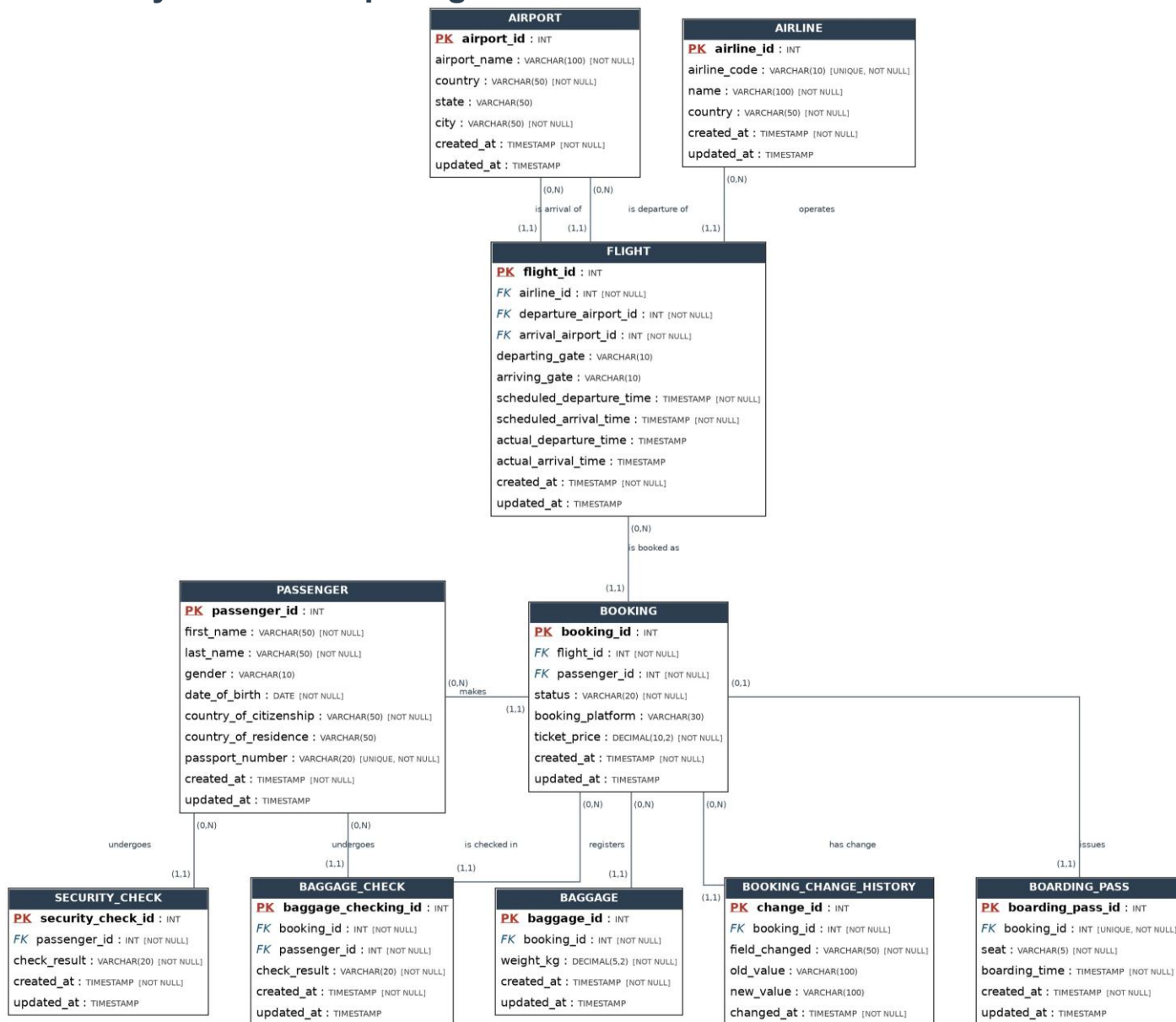
Entity A	Relationship	Entity B	Card. A	Card. B	Type
Airline	operates	Flight	(0,N)	(1,1)	One-to-Many
Airport	is departure airport of	Flight	(0,N)	(1,1)	One-to-Many
Airport	is arrival airport of	Flight	(0,N)	(1,1)	One-to-Many
Passenger	makes	Booking	(0,N)	(1,1)	One-to-Many
Flight	is booked as	Booking	(0,N)	(1,1)	One-to-Many
Booking	has change	Booking_Change_History	(0,N)	(1,1)	One-to-Many
Booking	issues	Boarding_Pass	(0,1)	(1,1)	One-to-One
Booking	registers	Baggage	(0,N)	(1,1)	One-to-Many
Booking	is checked in	Baggage_Check	(0,N)	(1,1)	One-to-Many
Passenger	undergoes	Baggage_Check	(0,N)	(1,1)	One-to-Many
Passenger	undergoes	Security_Check	(0,N)	(1,1)	One-to-Many

Relationship descriptions:

- **Airline – Flight** (operates): An airline may operate many flights; every flight belongs to exactly one airline.
- **Airport – Flight** (is departure airport of): An airport may be the departure point of many flights; every flight has exactly one departure airport.
- **Airport – Flight** (is arrival airport of): An airport may be the arrival point of many flights; every flight has exactly one arrival airport.
- **Passenger – Booking** (makes): A passenger may make many bookings; every booking belongs to exactly one passenger.
- **Flight – Booking** (is booked as): A flight may have many bookings; every booking is for exactly one flight.
- **Booking – Booking_Change_History** (has change): A booking may have many recorded changes; every change record belongs to exactly one booking.
- **Booking – Boarding_Pass** (issues): A booking may have at most one boarding pass; every boarding pass belongs to exactly one booking.
- **Booking – Baggage** (registers): A booking may register many baggage items (or none); every baggage item belongs to exactly one booking.
- **Booking – Baggage_Check** (is checked in): A booking may have many baggage checks; every baggage check belongs to exactly one booking.

- **Passenger – Baggage_Check** (undergoes): A passenger may undergo many baggage checks; every baggage check belongs to exactly one passenger.
- **Passenger – Security_Check** (undergoes): A passenger may undergo many security checks; every security check belongs to exactly one passenger.

4. Entity-Relationship Diagram



5. Legend / Notation

Symbol	Meaning
Rectangle (box)	An entity (table) in the database
PK	Primary Key – uniquely identifies each row of the entity
FK	Foreign Key – references the primary key of another entity
[NOT NULL]	Attribute is required (mandatory)

[UNIQUE]	Attribute values must be unique across the entity
Line between entities	A relationship between two entities
(1,1)	Mandatory participation, exactly one – every row on this side is related to exactly one row on the other side
(0,1)	Optional participation, at most one
(0,N)	Optional participation, many – zero or more related rows
Symbol	Meaning
(1,N)	Mandatory participation, many – one or more related rows

6. Design Notes and Assumptions

- Flight references Airport twice (departure and arrival), so the diagram shows two distinct relationships between the same pair of entities.
- Booking changes are stored in a separate Booking_Change_History table, as required by the task description, instead of overwriting the original booking values.
- Boarding_Pass has a one-to-one relationship with Booking: a booking is issued at most one boarding pass, and a boarding pass always belongs to exactly one booking.
- Baggage_Check is linked directly to both Booking and Passenger (as stated in the system description), rather than to an individual Baggage item, since the source description does not give baggage_id as an attribute of the check.
- All created_at attributes are required (set automatically on insert); all updated_at attributes are optional until the first update.
- Data types are chosen using generic SQL types (INT, VARCHAR(n), DATE, TIMESTAMP, DECIMAL(p,s)) suitable for implementation in PostgreSQL.